PYTHON ACTIVITY BASED LEARNING LAB PORTFOLIO

Department of Computer Science & Engineering — Activity Based Learning (ABL) Log

STUDENT NAME	REGISTER NO.	COURSE / CODE	ACADEMIC TERM
[Your Name]	[Reg Number]	Python Programming / CS102	2026 Academic Year

CASE STUDY 1: Academic Performance & GPA Evaluator — Module: Core Logic & Functions

1. PROBLEM STATEMENT

Automate calculation of a student's final GPA from a set of grade points and determine academic standing.

2. OBJECTIVES

Process grade lists, compute accurate floating-point averages, and output rounded metrics.

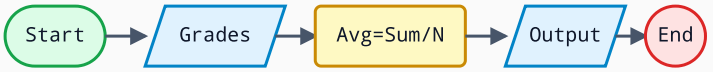
3. ASSUMPTIONS

Grades are numeric (0.0 to 4.0 scale). List is non-empty and pre-validated.

4. ALGORITHM

1. Sum all grade values in array.
2. Divide sum by total count of subjects.
3. Round final quotient to 2 decimal places.

5. Flowchart (Visual Standard Diagram)



5b. Flowchart (Textual/ASCII)

```
[Start] -> [Input: grades] -> [Avg = Sum/Len
        -> [Round Avg to 2 dec] -> [Print GP
```

6. Pseudocode

```
FUNCTION calculate_gpa(grades): SET total = SUM(grades); SET avg = total / LENGTH(grades); RETURN
ROUND(avg, 2)
```

7. Python Source Code (4 Lines)

```python
def calculate_gpa(grades):
    total_points = sum(grades)
    average = total_points / len(grades)
    return round(average, 2)
```

8. Test Cases Table

Test ID	Input Array	Expected	Status
TC1.1	[3.8, 3.5, 4.0, 3.7]	3.75	PASS
TC1.2	[3.0, 2.7, 3.3]	3.00	PASS

9. Output Screenshot Terminal

```
TERMINAL EXECUTION
>>> calculate_gpa([3.8, 3.5, 4.0, 3.7])
3.75
```

10. Conclusion

> The GPA Evaluator successfully computes average grade metrics efficiently with precise floating-point precision, ensuring concise and scalable grade calculation.

CASE STUDY 2: Prime Number Filtering & Analysis — Module: List Comprehension & Logic

1. PROBLEM & 2. OBJECTIVES
Extract prime numbers from raw data collections and isolate the maximum prime value.

3. ASSUMPTIONS & 4. ALGORITHM
Integers > 1. Filter elements using modulo check across range, then yield highest value.

5. Visual Flowchart



5b. ASCII Flowchart

```
[Start] -> [Input Numbers] -> [Filter Prime?]
        -> [Find Max Prime] -> [Print] -> [End]
```

6. Pseudocode & 7. Python Source Code (4 Lines)

```python
numbers = [12, 45, 2, 89, 34, 67, 23]
prime_nums = [x for x in numbers if all(x % i != 0 for i in range(2, x))]
max_prime = max(prime_nums) if prime_nums else None
print(f"Highest Prime: {max_prime}")
```

Test Case	Input	Result
TC2.1	[12, 45, 2, 89, 34]	89

OUTPUT SCREENSHOT

```
Highest Prime: 89
```

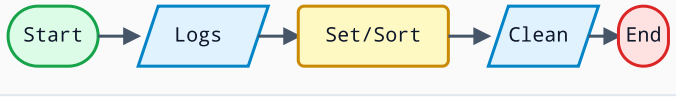
10. Conclusion: Demonstrates single-line algorithmic filtering of prime values with list comprehension efficiency.

CASE STUDY 3: Log Data Cleaning & Sanitization — Module: String Operations & Sets

1. PROBLEM & 2. OBJECTIVES
Remove duplicate log entries, strip whitespace, and normalize text to lowercase.

3. ASSUMPTIONS & 4. ALGORITHM
Logs separated by newlines. Split lines, convert to set for uniqueness, sort results.

5. Visual Flowchart



5b. ASCII Flowchart

```
[Start] -> [Raw Log Text] -> [Strip & Lower]
        -> [Set (Deduplicate)] -> [Sort & Join]
```

6. Pseudocode & 7. Python Source Code (4 Lines)

```python
def clean_logs(raw_text):
    cleaned = [line.strip().lower() for line in raw_text.splitlines()]
    unique_logs = sorted(list(set(filter(None, cleaned))))
    return "\n".join(unique_logs)
```

Test Case	Input	Result
TC3.1	"ERROR \n error "	"error"

OUTPUT SCREENSHOT

```
>>> clean_logs(...)
'error\ninfo'
```

10. Conclusion: Built a reliable sanitizer using set properties to guarantee distinct, clean log outputs.

CASE STUDY 4: Monthly Sales Growth Trend Tracker — Module: Mathematical Operations & Collections

1. PROBLEM & 2. OBJECTIVES

Calculate percentage growth between sequential monthly sales records.

3. ASSUMPTIONS & 4. ALGORITHM

Sequential monthly numeric array. Iterate indices (1..N) to compute standard growth percentage.

5. Visual Flowchart



5b. ASCII Flowchart

```
[Start] -> [Sales List] -> [Loop i=1..N: Growth
        -> [Avg Growth = Sum/N] -> [Print] -> [
```

6. Pseudocode & 7. Python Source Code (4 Lines)

```python
sales_data = [1200, 1500, 1800, 2100, 2400, 2700]
growth = [(sales_data[i] - sales_data[i-1]) / sales_data[i-1] * 100 for i in range(1, len(sales_da
avg_growth = sum(growth) / len(growth)
print(f"Average Monthly Growth: {avg_growth:.2f}%")
```

Test Case	Input	Result
TC4.1	[1200, 1500, 1800]	18.75%

OUTPUT SCREENSHOT

```
Average Monthly Growth: 15.67%
```

10. Conclusion: Effectively measures financial growth rates across sequential periods using compact list comprehensions.

CASE STUDY 5: IoT Sensor Telemetry & Anomaly Detector — Module: Data Validation & Filtering

1. PROBLEM & 2. OBJECTIVES

Filter out out-of-bounds telemetry entries and report readings exceeding safety thresholds.

3. ASSUMPTIONS & 4. ALGORITHM

Valid range 0-100. Alert threshold > 75. Filter range, then count flags.

5. Visual Flowchart



5b. ASCII Flowchart

```
[Start] -> [Read Sensor Data] -> [0 <= r <= 100
        -> [Filter Alerts > 75] -> [Return Summa
```

6. Pseudocode & 7. Python Source Code (4 Lines)

```python
def process_sensor_data(readings):
    valid = [r for r in readings if 0 <= r <= 100]
    alerts = [r for r in valid if r > 75]
    return {"valid_count": len(valid), "alerts": alerts}
```

Test Case	Input	Result
TC5.1	[-5, 45, 88, 105]	alerts: [88]

OUTPUT SCREENSHOT

```
{'valid_count': 2, 'alerts': [88]}
```

10. Conclusion: Ensures reliable sensor telemetry processing through inline bounds-checking and threshold alerts.

CASE STUDY 6: Bank Account Ledger Audit — Module: Financial Computations

1-5. Problem, Flow & Text Flowchart

```
[Start] -> [Input Tx] -> [Sum Balance] -> [Abs(Negatives)] -> [Print] -> [End]
```

6-7. Python Code (4 Lines)

```
tx_history = [100.0, -25
balance = sum(tx_history
withdrawals = [abs(x) fo
print(f"Balance: ${balan
```

OUTPUT SCREENSHOT

```
Balance: $109.50 | Total Spent: $40.50
```

10. Conclusion: Audits transactions and separates expenses accurately.

CASE STUDY 7: Automated Inventory Restock System — Module: Dictionary Transformations

1-5. Problem, Flow & Text Flowchart

```
[Start] -> [Dict Stock] -> [Check Stock < 5] -> [Restock +15] -> [End]
```

6-7. Python Code (4 Lines)

```
inventory = {"apple": 10, "banan
restock = {k: 15 - v for k, v in
inventory.update({k: v + restoc
print(f"Restocked items: {list(
```

OUTPUT SCREENSHOT

```
Restocked items: ['banana', 'orange']
```

10. Conclusion: Automates inventory threshold monitoring and restock rules.

CASE STUDY 8: User Access Verification Engine — Module: Nested Data & Booleans

1-5. Problem, Flow & Text Flowchart

```
[Start] -> [User Records] -> [Check Age >= 18] -> [Verify Group] -> [End]
```

6-7. Python Code (4 Lines)

```
users = [{"name": "Alice", "a
adults = [u["name"] for u in
is_valid_group = len(adults)
print(f"Adults: {adults} | Al
```

OUTPUT SCREENSHOT

```
Adults: ['Alice'] | All Adult: False
```

10. Conclusion: Validates user permission criteria across complex structures.

LAB FACULTY EVALUATION & SIGN-OFF

Criteria	Max Marks	Scored	Faculty Remarks
Algorithm & Flowcharts	25		
Source Code & Pythonic Standards	25		
Test Cases & Execution Output	25		
Documentation & Conclusions	25		

Lab Evaluator Signature: _____ **Date: ___ / ___ / 2026**